

Week 1 – PLSQL_Exercises

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Code :

Scenario 1: Apply 1% Discount to Interest Rates for Customers Above 60

DECLARE

CURSOR c_customers IS

SELECT customer_id, age

FROM customers;

BEGIN

FOR rec IN c_customers LOOP

IF rec.age > 60 THEN

UPDATE loans

SET interest_rate = interest_rate - (interest_rate * 0.01)

WHERE customer_id = rec.customer_id;

END IF;

END LOOP;

```
COMMIT;  
  
DBMS_OUTPUT.PUT_LINE('Discount applied to eligible customers.');
```

END;

/

Scenario 2: Set IsVIP = TRUE for Customers with Balance > 10000

```
BEGIN
```

```
FOR c IN (
```

```
    SELECT CustomerID, Balance
```

```
    FROM Customers
```

```
) LOOP
```

```
    IF c.Balance > 10000 THEN
```

```
        UPDATE Customers
```

```
        SET IsVIP = 'TRUE'
```

```
        WHERE CustomerID = c.CustomerID;
```

```
    END IF;
```

```
END LOOP;
```

```
COMMIT;
```

```
END;
```

/

Scenario 3: Send Reminder for Loans Due Within Next 30 Days

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
FOR l IN (
```

```
    SELECT CustomerID, LoanID, DueDate
```

```

FROM Loans
WHERE DueDate BETWEEN SYSDATE AND SYSDATE + 30
) LOOP
    DBMS_OUTPUT.PUT_LINE(
        'Reminder: Customer ' || I.CustomerID ||
        ' has Loan ' || I.LoanID ||
        ' due on ' || TO_CHAR(I.DueDate, 'DD-MON-YYYY')
    );
END LOOP;
END;
/

```

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Scenario 1: Process Monthly Interest

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS

    CURSOR c_savings IS
        SELECT account_id, balance
        FROM accounts
        WHERE account_type = 'SAVINGS';

BEGIN
    FOR rec IN c_savings LOOP
        UPDATE accounts
        SET balance = balance + (balance * 0.01)
        WHERE account_id = rec.account_id;

        DBMS_OUTPUT.PUT_LINE(
            'Account ID: ' || rec.account_id ||
            ' | Old Balance: ' || rec.balance ||
            ' | New Balance: ' || ROUND(rec.balance + (rec.balance * 0.01), 2)
        );
    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Monthly interest processed for all savings
accounts.');
```

END ProcessMonthlyInterest;

/

Scenario 2 – UpdateEmployeeBonus

```

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_department_id IN NUMBER,
    p_bonus_percent IN NUMBER
) IS
    CURSOR c_employees IS
        SELECT employee_id, employee_name, salary
        FROM employees
        WHERE department_id = p_department_id;
BEGIN
    FOR rec IN c_employees LOOP
        UPDATE employees
        SET salary = salary + (salary * p_bonus_percent / 100)
        WHERE employee_id = rec.employee_id;

        DBMS_OUTPUT.PUT_LINE(
            'Employee: ' || rec.employee_name ||
            ' | Old Salary: ' || rec.salary ||
            ' | New Salary: ' || ROUND(rec.salary + (rec.salary * p_bonus_percent /
100), 2)
        );
    END LOOP;
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Bonus updated for Department ID: ' ||
p_department_id);
END UpdateEmployeeBonus;
/

```

Scenario 3: Transfer Funds Between Accounts

```
CREATE OR REPLACE PROCEDURE TransferFunds (  
    p_from_account IN NUMBER,  
    p_to_account   IN NUMBER,  
    p_amount       IN NUMBER  
) IS  
    v_balance NUMBER;  
  
BEGIN  
    -- Check source account balance  
    SELECT balance INTO v_balance  
    FROM accounts  
    WHERE account_id = p_from_account;  
  
    -- Validate sufficient funds  
    IF v_balance < p_amount THEN  
        DBMS_OUTPUT.PUT_LINE(  
            'Transfer FAILED: Insufficient balance in Account ' ||  
            p_from_account ||  
            '. Available: ' || v_balance  
        );  
    ELSE  
        -- Deduct from source  
        UPDATE accounts  
        SET balance = balance - p_amount  
        WHERE account_id = p_from_account;
```

```

-- Add to destination

UPDATE accounts

SET balance = balance + p_amount

WHERE account_id = p_to_account;


COMMIT;

DBMS_OUTPUT.PUT_LINE(
    'Transfer SUCCESSFUL: Rs.' || p_amount ||
    ' transferred from Account ' || p_from_account ||
    ' to Account ' || p_to_account
);

END IF;


EXCEPTION

WHEN NO_DATA_FOUND THEN

    DBMS_OUTPUT.PUT_LINE('ERROR: Account not found.');
```

WHEN OTHERS THEN

```

    ROLLBACK;

    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

END TransferFunds;

/
```